Reterna

Recursive Functions With
Ternary Conditional Operator

▸ Reterna Syntax

⟨Programm⟩ ::= ε | ⟨Funktion⟩ "↲" ⟨Programm⟩
⟨Funktion⟩ ::= ⟨Funktionsname⟩ "(" ⟨PListe⟩ ") =" ⟨Ausdruck⟩ ⇦ *Funktionsdefinition*
⟨PListe⟩ ::= ⟨Parameter⟩ | ⟨Parameter⟩ "," ⟨PListe⟩
⟨Ausdruck⟩ ::=
⟨Parameter⟩ |
⟨Konstante⟩ |
⟨Ausdruck⟩ "+1" |
⟨Ausdruck⟩ "−1" |
⟨Funktionsname⟩ "(" ⟨AListe⟩ ")" | ⇦ *Funktionsaufruf*
⟨Ausdruck⟩ "?" ⟨Ausdruck⟩ ":" ⟨Ausdruck⟩ | ⇦ *Ternary Conditional Operator*
"(" ⟨Ausdruck⟩ ")"
⟨AListe⟩ ::= ⟨Ausdruck⟩ | ⟨Ausdruck⟩ "," ⟨AListe⟩

⟨Funktionsname⟩ und ⟨Parameter⟩: String aus Buchstaben und Ziffern
⟨Konstante⟩: Zahl ∈ $\mathbb{N}_0$

▸ Werte

Parameter enhalten Werte ∈ $\mathbb{N}_0$ = {0, 1, 2, …}
Funktionen berechnen Abbildungen $\mathbb{N}_0^k \rightarrow \mathbb{N}_0$

▸ 0 ist untere Grenze

Wenn der Parameter n den Wert 0 hat,
dann ergibt n−1 ebenfalls den Wert 0

▸ Wahrheitswerte

false = 0
true = 1
alle Werte >1 werden als *true* interpretiert

▸ Beispiele

`zero(n) = 0`

`zero(n) = n ? zero(n−1) : n`

`identity(n) = n`

`add1(n) = n+1`

`add3(n) = add1(add1(n+1))`

oder auch:

`add3(n) = ((n+1)+1)+1`

▸ Ternary Conditional Operator

a ? b : c

if a *then* b *else* c

⟨Ausdruck⟩ "?" ⟨Ausdruck⟩ ":" ⟨Ausdruck⟩

- ⟨Ausdruck⟩: Bedingung
- ⟨Ausdruck⟩: wenn Bedingung *true*
- ⟨Ausdruck⟩: wenn Bedingung *false*

Beispiel

`not(n) = n ? false : true`

not(0) = not(*false*) = *true*
not(12) = not(*true*) = *false*

▸ Beispiel: Finbonacci-Zahlen

fib(0) = fib(1) = 1, und für alle n ≥ 2 gilt:
fib(n) = fib(n−1) + fib(n−2)

`fib(n) = n−1 ? fib(n−1) + fib((n−1)−1) : 1`

▸ Anmerkung: Ternary Operator hat niedrigste Priorität

`n + 1 ? a : b` = `(n + 1) ? a : b`

≠ `n + (1 ? a : b)`

Rechenoperationen

▸ Addition

n + m — `add(n, m) = m ? add(n+1, m−1) : n`

Beispiel

add(5, 3) = add(6, 2) = add(7, 1) = add(8, 0) = 8

▸ Subtraktion

n − m — `sub(n, m) = m ? sub(n−1, m−1) : n`

▸ Multiplikation

n · m — `mul(n, m) = m ? mul(n, m−1) + n : 0`

$$n \cdot m = \begin{cases} n \cdot (m-1) + n, & \text{wenn } m > 0 \\ 0, & \text{sonst} \end{cases}$$

Beispiel

mul(5, 3) = mul(5, 2) + 5 = 10 + 5 = 15
mul(5, 2) = mul(5, 1) + 5 = 5 + 5 = 10
mul(5, 1) = mul(5, 0) + 5 = 0 + 5 = 5
mul(5, 0) = 0

▸ Division und Modulo

n : m	div(n, m) = (n+1) − m ? div(n − m, m) + 1 : 0
n % m	mod(n, m) = (n+1) − m ? mod(n − m, m) : n

▸ Exponentiation

n ^ m	pow(n, m) = powA(n, m, 1) powA(n, m, p) = m ? powA(n, m−1, p · n) : p

powA(n, m, p) = $p \cdot n^m$

Trick: Ergebnis der Rechnung in einem extra-Argument ("Akkumulator") zusammenbauen

Beispiel

pow(4, 3) = powA(4, 3, 1)
= powA(4, 2, 4)
= powA(4, 1, 16)
= powA(4, 0, 64) = 64

▸ logische Operationen

n *and* m	and(n, m) = n ? m : *false*
n *or* m	or(n, m) = n ? *true* : m
not n	not(n) = n ? *false* : *true*

▸ Vergleiche

n > m	greater(n, m) = n − m ? *true* : *false*
n ≥ m	greaterEq(n, m) = greater(n+1, m)
n < m	less(n, m) = greater(m, n)
n ≤ m	lessEq(n, m) = greaterEq(m, n)
n = m	equal(n, m) = (n ≥ m) *and* (n ≤ m)

$n \neq m$ | notEqual(n, m) = *not* equal(n, m)

Strings

Alphabet $\Sigma = \{c_1, c_2, \ldots, c_m\}$
$code(c_j) = j$ für alle $j \in \{1, 2, \ldots, m\}$

für $a_0a_1\ldots a_n \in \Sigma^*$:

$$code(a_0a_1\ldots a_n) = \sum_{i=0}^{n} code(a_i) \cdot m^i$$

$code(\varepsilon) = 0$

▸ Notation

schreibe z.B. "A" statt code("A")
schreibe z.B. "ABCD" statt code("ABCD")

▸ Hilfsfunktionen

$code(a_1a_2\ldots a_n) = (code(a_0a_1a_2\ldots a_n) - 1) / m$

restStr(a) = (a−1) / m

$code(a_0) = (code(a_0a_1a_2\ldots a_n) - 1) \% m + 1$

firstChar(a) = ((a−1) % m)+1

▸ das i-te Zeichen in einem String

a[i] | charAt(a, i) = i ? charAt(restStr(a), i−1) : firstChar(a)

▸ Länge eines Strings

len(a) | len(a) = a ? len(restStr(a))+1 : 0

▸ Akzeptieren von Sprachen

$L = \{a \in \Sigma^* \mid a \text{ ist ein Palindrom}\}$

```
palindrom(a) = palindrom2(a, 0, len(a)−1)
palindrom2(a, i, j) =
      i < j ? (a[i] = a[j]) and palindrom2(a, i+1, j−1) : true
```

$L = \{A^nB^nC^n \mid n \in \mathbb{N}_0\}$

```
AnBnCn(a) = len(a) ? AnBnCnImp(a, len(a)) : true
AnBnCnImp(a, n) =
    n % 3 ? false : testChars(a, 0, n/3, "A") and
                    testChars(a, n/3, 2·n/3, "B") and
                    testChars(a, 2·n/3, n, "C")
testChars(a, i, j, c) =
    i < j ? (a[i] = c) and testChars(a, i+1, j, c) : true
```

▸ Ackermann-Funktion

$A_k(n)$

k: Stufe

n: Eingabegröße

Für $k \in \mathbb{N}$ und $n \in \mathbb{N}$ mit $n \geq 2$ gelte:

$A_0(1) = 2$
$A_k(1) = 2$
$A_0(n) = n+2$
$A_k(n) = A_{k-1}(A_k(n-1))$

```
A(k, n) = k ? ackerk(k, n) : acker0(n)
acker0(n) = n > 1 ? n + 2 : 2
ackerk(k, n) = n > 1 ? A(k−1, A(k, n−1)) : 2
```